INTRODUCCIÓN A JAVA

Programación



PARTE III

INDICE

FUNCIONES **3**

DECLARACIÓN DE UNA FUNCIÓN 3
LLAMADA A UNA FUNCIÓN 7
DEVOLUCIÓN DE UN VALOR 9
ÁMBITO DE LAS VARIABLES 10
PROCEDIMIENTOS, FUNCIONES Y MÉTODOS 10
PARÁMETROS: PASO POR VALOR Y POR REFERENCIA 10

EXCEPCIONES **13**

JERARQUÍA Y TIPOS DE EXCEPCIONES JAVA 15
MANEJAR EXCEPCIONES (TRY – CATCH – FINALLY) 17
MANEJADORES DE EXCEPCIONES *17*
PARTICULARIDADES DE LA CLÁUSULA CATCH *19*
CLÁUSULAS CATCH MÚLTIPLES *19*
EL OBJETO EXCEPTION *21*
LANZAR EXCEPCIONES (THROW) 21
¿POR QUÉ LANZAR EXCEPCIONES? *21*
CÓMO LANZAR UNA EXCEPCIÓN *22*
INDICAR LA EXCEPCIÓN EN LA CABECERA DEL MÉTODO *23*
LANZANDO DISTINTOS TIPOS DE EXCEPCIONES *23*
LAS EXCEPCIONES SE LANZAN DE MÉTODO A MÉTODO *24*
DEFINIR EXCEPCIONES PROPIAS 25

FUNCIONES

La mejor forma de crear y mantener un programa grande es construirlo a partir de piezas más pequeñas o módulos. Cada uno de los cuales es más manejable que el programa en su totalidad.

Las **funciones** (**subprogramas**) son **utilizadas para evitar la repetición de código** en un programa al poder ejecutarlo desde varios puntos de un programa con sólo invocarlo.

El concepto de función es una forma de encapsular un conjunto de instrucciones dentro de una declaración específica (llamada generalmente SUBPROGRAMA o FUNCIÓN), permitiendo la descomposición funcional y la diferenciación de tareas.

Utilidad principal de las funciones:

- Agrupar código que forma una entidad propia o una idea concreta.
- Agrupar código que se necesitará varias veces en un programa, con al misión de no repetir código.
- Dividir el código de un programa grande un subprogramas (funciones), cada uno de ellos especializados en resolver una parte del problema.

Características de las funciones:

- Se definen mediante un nombre único que representa el bloque de código.
- Pueden ser llamadas (ejecutadas) desde cualquier parte del código.
- Se les puede pasar valores para que los procesen de alguna forma.
- Pueden devolver un resultado para ser usado desde donde se les llamó.

DECLARACIÓN DE UNA FUNCIÓN

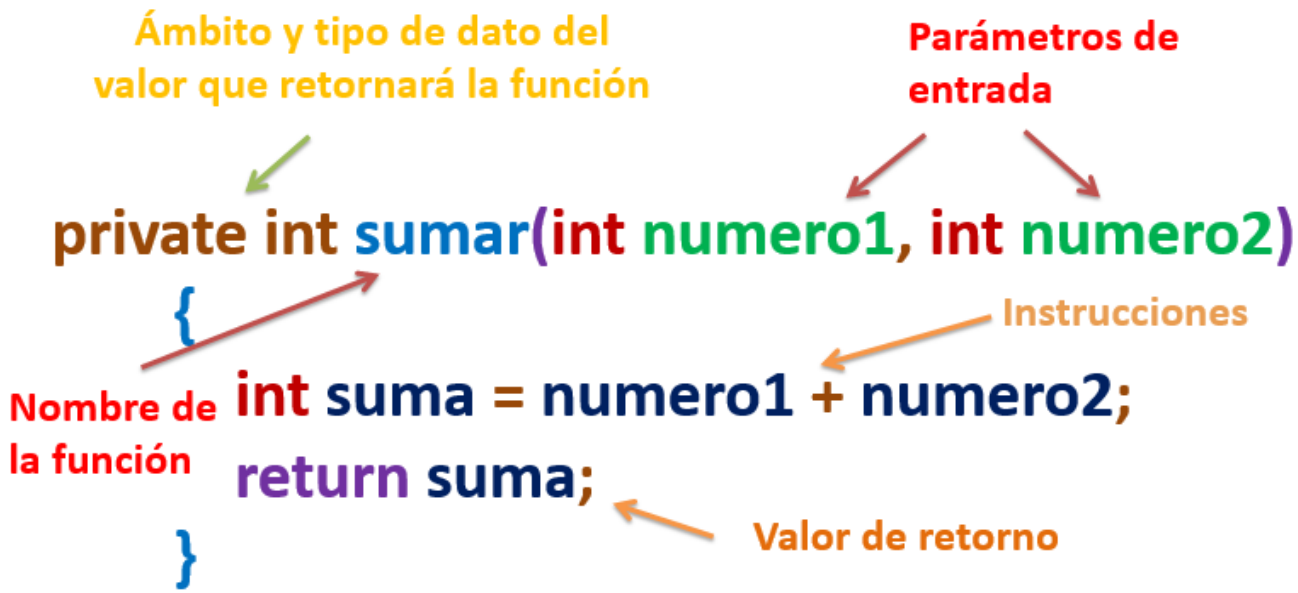
Declarar una función simplemente significa crearla para que luego pueda ser llamada (utilizada) desde otro lugar del código de nuestro programa. Una función se estructura en **cabecera** y **cuerpo**.

La **cabecera** se declara en una sola línea y se compone de:

- **Modificadores de función**: Existen muchos pero los veremos en futuras unidades. Por ahora solo utilizaremos public static).
- **Tipo devuelto**: El tipo de dato que devolverá la función, como por ejemplo int, double, char, boolean, String, etc. Si la función no devuelve nada se indica mediante void.
- **Nombre de la función**: Identificador único para llamar a la función.
- **Lista de parámetros**: Indica los tipos y nombres de los datos que se le pasarán a la función cuando sea llamada. Pueden ser varios o ninguno.

El **cuerpo** es un bloque de código entre llaves { … } que se ejecutará cuando desde otra parte del código utilicemos la función.

```
[Modif_de_función] Tipo_devuelto Nombre_de_función (lista_de_parámetros)
{
        …
}
```



Ejemplos de funciones:

```
public static void imprimeHolaMundo() {
        System.out.println("Hola mundo");
}
```

Este es un ejemplo muy sencillo de una función llamada 'imprimeHolaMundo', que no tiene parámetros de entrada (no hay nada entre los paréntesis) y no devuelve ningún valor (indicado por void). Cuando la llamemos lo único que hará será escribir por pantalla el mensaje "Hola mundo".

```
public static void imprimeHolaNombre(String nombre) {
        System.out.println("Hola " + nombre);
}
```

Esta función se llama 'imprimeHolaNombre', tiene como parámetro de entrada un dato String llamado 'nombre' y no devuelve nada. Cuando la llamemos imprimirá por pantalla el texto "Hola " seguido del String nombre que se le pase como parámetro.

```
public static int doble(int a) {
        int resultado = a * 2;
        return resultado;
}
```

Esta función se llama 'doble', tiene como parámetro de entrada un dato int llamado 'a' y devuelve un dato de tipo int. Cuando la llamemos calculará el doble de 'a' y lo devolverá (con el return).

```
public static int multiplica(int a, int b) {
        int resultado = a * b;
        return resultado;
}
```

Esta función se llama 'multiplica', tiene dos parámetros de entrada de tipo int llamados 'a' y 'b' y devuelve un dato de tipo int. Cuando la llamemos calculará a*b y lo devolverá (con el return).

```
public static double maximo(double valor1, double valor2) {

        double max;

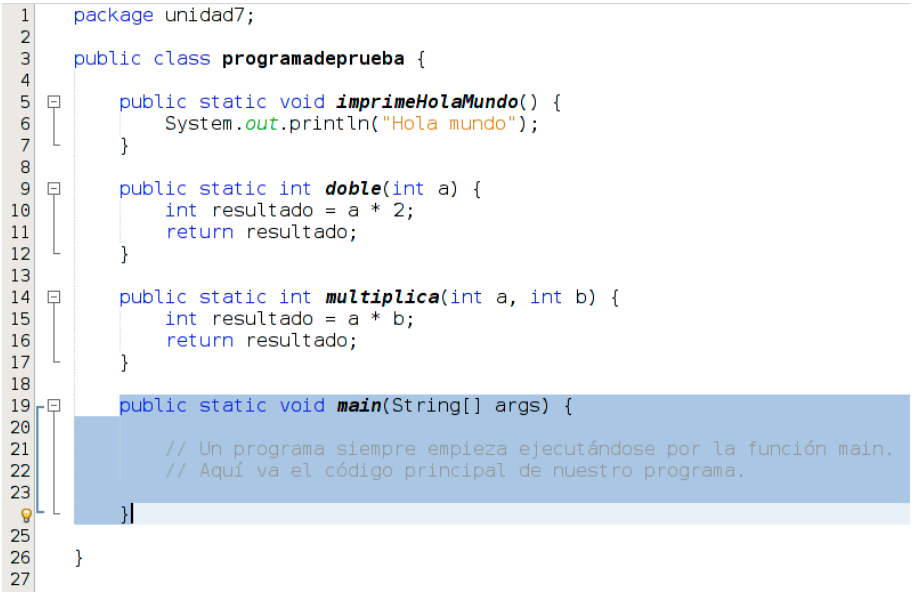
        if (valor1 > valor2)
                max = valor1;
        else
                max = valor2;

        return max;

}
```

Esta función se llama 'maximo', tiene dos parámetros de entrada de tipo double llamados 'valor1' y 'valor2' y devuelve un dato de tipo double. Cuando la llamemos calculará el máximo entre 'valor1' y 'valor2' y lo devolverá.

Es importante saber que las funciones se declaran dentro de 'class' pero fuera del 'main'.



```
1  package unidad7;
2
3  public class programadeprueba {
4
5      public static void imprimeHolaMundo() {
6          System.out.println("Hola mundo");
7      }
8
9      public static int doble(int a) {
10         int resultado = a * 2;
11         return resultado;
12     }
13
14     public static int multiplica(int a, int b) {
15         int resultado = a * b;
16         return resultado;
17     }
18
19     public static void main(String[] args) {
20
21         // Un programa siempre empieza ejecutándose por la función main.
22         // Aquí va el código principal de nuestro programa.
23
24     }
25
26 }
27
```

En este programa tenemos 4 funciones: imprimeHolaMundo, doble, multiplica y main. Sí, el 'main' donde siempre has programando hasta ahora es en efecto una función, pero un poco especial: '**main' es** la función principal, **el punto de inicio de un programa.**

Es obligatorio que todo programa Java tenga una función main. Si te fijas, es una función que recibe como parámetro un String[] (vector de String) y no devuelve nada (aunque podría devolver un int). El por qué de esto lo veremos más adelante.

Las 3 funciones que hemos declarado arriba del main por sí solas no hacen nada, simplemente están ahí esperando a que sean llamadas (utilizadas), normalmente desde el propio main.

Vamos a verlo.

LLAMADA A UNA FUNCIÓN

Las funciones pueden ser invocadas o llamadas desde cualquier otra función, incluida ella misma.

Sí, una función puede llamar a cualquier otra función, y una función puede llamarse a sí misma.

De todos modos **por ahora llamaremos funciones solo desde la función principal 'main'**. Así es más sencillo de aprender al principio.

Cuando se invoca una función el flujo de ejecución salta a la función (pasándole los parámetros si los hubiera), se ejecutan las instrucciones de la función y por último vuelve al punto que llamó a la función para seguir ejecutándose.

Las funciones se invocan con su nombre, pasando la lista de parámetros entre paréntesis. Si no tiene parámetros han de ponerse los paréntesis igualmente. Si la función devuelve un valor, para recogerlo hay que asignarlo a una variable o utilizarlo de algún modo (pueden combinarse funciones en expresiones e incluso pasarlo a otras funciones).

Ejemplo utilizando las funciones del apartado anterior:

```
public static void main(String[] args) {

    // No tiene parámetros ni devuelve valor.  Simplemente imprime "Hola Mundo"
    imprimeHolaMundo();


    // Es habitual llamar a una función y guardar el valor devuelto en una variable
    int a = doble(10);              // a = 20 (10*2)
    int b = multiplica(3, 5);       // b = 15 (3*5)


    // Pueden pasarse variables como parámetros
    int c = doble(a);               // c = 40 (20*2)
    int d = multiplica(a, b);       // d = 300 (20*15)
```

```
        // Pueden combinarse funciones y expresiones
        int e = doble(4) + multiplica(2,10);      // e = 8 + 20
        System.out.println("El doble de 35 es " + doble(35) );  // "El doble de 35 es 70"
        System.out.println("12 por 12 es " + multiplica(12,12) );  // "12 por 12 es 144"
}
```

Ejemplo: *Programa con una función que suma dos números*.

```
public class Suma {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2, suma;

        System.out.print("Introduce un número: ");
        num1 = sc.nextInt();

        System.out.print("Introduce otro número: ");
        num2 = sc.nextInt();

        suma = suma(num1, num2);

        System.out.println("La suma es: " + suma);

    }

    public static int suma(int n1, int n2) {

        int suma;

        suma = n1 + n2;

        return suma;
    }

}
```

Salida:

```
run:
Introduce un número: 3
Introduce otro número: 4
La suma es: 7
BUILD SUCCESSFUL (total time: 3 seconds)
```

Ejemplo: *Programa con una función que determina si un número es par o impar*.

```
public class ParImpar {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int num;

        System.out.print("Introduce un número: ");
        num = in.nextInt();

        if(par(num) == true) // Llamada a la función desde la expresión
            System.out.println(num + " es par.");
        else
            System.out.println(num + " es impar.");
    }

    public static boolean par(int numero)
    {
        boolean par = false;

        if(numero % 2 == 0) // Si el resto es 0 par será 'true' sino 'false'
            par = true;

        return par;
    }
}
```

Salida:



```
run:
Introduce un número: 9
9 es impar.
BUILD SUCCESSFUL (total time: 2 seconds)
```

DEVOLUCIÓN DE UN VALOR

Los métodos pueden devolver valores de tipo básico o primitivo (int, double, boolean, etc.) **y también de tipo objeto** (Strings, arrays, etc.).

En todos los casos es el comando return el que realiza esta labor. En el caso de arrays y objetos, devuelve una referencia a ese array u objeto.

ÁMBITO DE LAS VARIABLES

Una función solo puede utilizar las variables de ámbito local, es decir, sus propias variables (los parámetros de la cabecera y las variables creadas dentro de la función). Cuando una función se ejecuta se crean sus variables, se utilizan y cuando la función termina se destruyen las variables.

Por todo ello una función no puede utilizar variables que estén fuera de ella, y fuera de una función no es posible utilizar variables de la propia función. A esta característica se le llama encapsulación y permite que las funciones sean independientes entre sí, facilitando el diseño de programas grandes y complejos.

Técnicamente sí es posible que una función utilice variables que están fuera de ella, pero eso lo veremos en futuras unidades cuando aprendamos Programación Orientada a Objetos.

PROCEDIMIENTOS, FUNCIONES y MÉTODOS

Es muy común entre programadores que se hable indistintamente de estos tres términos sin embargo poseen deferencias.

FUNCIONES → Las funciones son un conjunto de líneas de código (instrucciones), encapsulados en un bloque, usualmente reciben parámetros, cuyos valores utilizan para efectuar operaciones y adicionalmente retornan un valor con la ejecución return.

PROCEDIMIENTOS → Los procedimientos son básicamente un conjunto de instrucciones que se ejecutan sin retornar ningún valor.

MÉTODOS → Un método también puede recibir valores, efectuar operaciones con estos y retornar valores, sin embargo un método está asociado a un objeto, SIEMPRE, básicamente un método es una función que pertenece a un objeto o clase.

PARÁMETROS: PASO POR VALOR Y POR REFERENCIA

Existen dos tipos de parámetros y es importante comprender la diferencia.

- Parámetros de tipo simple (**paso por valor**): Como int, double, boolean, char, etc. En este caso se pasan por valor. Es decir, el valor se copia al parámetro y por lo tanto si se modifica dentro de la función esto no afectará al valor fuera de ella porque son variables distintas.

```
public static void main(String[] args) {
        int a = 10;
        System.out.println("Valor inicial de a: " + a); // a vale 10
        imprime_doble(a); // Se le pasa el 10 a la función
        System.out.println("Valor final de a: " + a); // a sigue valiendo 10
}

// El parámetro 'a' es independiente de la 'a' del main. ¡Son variables distintas!
public static void imprime_doble(int a) { // Se copia el valor 10 a esta nueva 'a'
        a = 2 * a; // Se duplica el valor de la 'a' de esta función, no afecta fuera
        System.out.println("Valor de a en la función: " + a); // 'a' vale 20
}
```

SALIDA:

```
run:
Valor inicial de a: 10
Valor de a en la función: 20
Valor final de a: 10
```

- Parámetros de tipo objeto (**paso por referencias**) : Como objetos de tipo String, los Arrays, etc. En este caso no se copia el objeto sino que se le pasa a la función una referencia al objeto original (un puntero). Por ello desde la función se accede directamente al objeto que se encuentra fuera. Los cambios que hagamos dentro de la función afectarán al objeto.

```
// Suma x a todos los elementos del vector v
public static void suma_x_al_vector(int v[], int x) {
        for (int i = 0; i< v.length; i++)
                v[i] = v[i] + x;
}

public static void main(String[] args) {
        int v[] = {0, 1, 2, 3};
        System.out.println("Vector antes: " + Arrays.toString(v));
        suma_x_al_vector(v, 10);
        System.out.println("Vector después: " + Arrays.toString(v));
}
```

SALIDA:

```
run:
Vector antes: [0, 1, 2, 3]
Vector después: [10, 11, 12, 13]
```

EXCEPCIONES

Una excepción es un error semántico que se produce en tiempo de ejecución.

Aunque un código sea correcto sintácticamente (es código Java válido y puede compilarse), es posible que durante su ejecución se produzcan errores inesperados, como por ejemplo:

- Dividir por cero.
- Intentar acceder a una posición de un array fuera de sus límites.
- Al llamar al nextInt() de un Scanner el usuario no introduce un valor entero.
- Intentar acceder a un fichero que no existe o que está en un disco duro corrupto.
- Etc.

Cuando esto ocurre, la máquina virtual Java crea un objeto de la clase **Exception** (las excepciones en Java son objetos) y se notifica el hecho al sistema de ejecución. Se dice que se ha lanzado una excepción ("Throwing Exception"). Existen también los errores internos que son objetos de la clase Error que no estudiaremos. Ambas clases Error y Exception son clases derivadas de la clase base Throwable.

Un método se dice que es capaz de tratar una excepción ("Catch Exception") si ha previsto el error que se ha producido y las operaciones a realizar para "recuperar" el programa de ese estado de error. No es suficiente capturar la excepción, si el error no se trata tan solo conseguiremos que el programa no se pare, pero el error puede provocar que los datos o la ejecución no sean correctos.

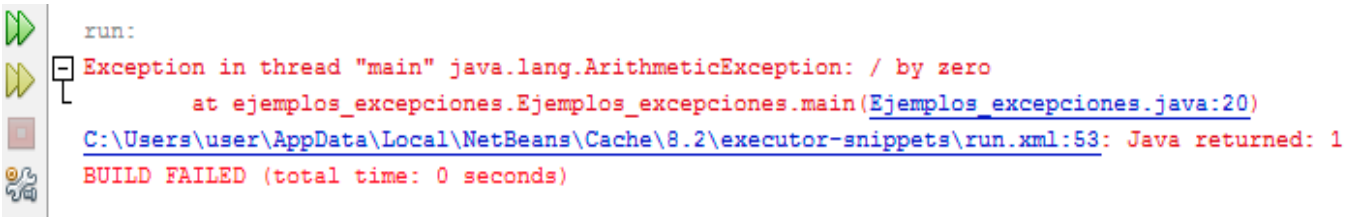
En el momento en que es lanzada una excepción, la máquina virtual Java recorre la pila de llamadas de métodos en busca de alguno que sea capaz de tratar la clase de excepción lanzada. Para ello, comienza examinando el método donde se ha producido la excepción; si este método no es capaz de tratarla, examina el método desde el que se realizó la llamada al método donde se produjo la excepción y así sucesivamente hasta llegar al último de ellos. En caso de que ninguno de los métodos de la pila sea capaz de tratar la excepción, la máquina virtual Java muestra un mensaje de error y el programa termina.

Los programas escritos en Java también pueden lanzar excepciones explícitamente mediante la instrucción **throw**, lo que facilita la devolución de un "código de error" al método que invocó el método que causó el error.

Ejemplo: Como primer encuentro con las excepciones, vamos a ejecutar el siguiente programa. En él vamos a forzar una excepción al intentar dividir un número entre 0.

```
12  public class Ejemplos_excepciones {
13
14      public static void main(String[] args) {
15          int div, x, y;
16
17          x = 3;
18          y = 0;
19
20          div = x / y;
21
22          System.out.println("El resultado es " + div);
23      }
24
25  }
```

Siendo la salida:

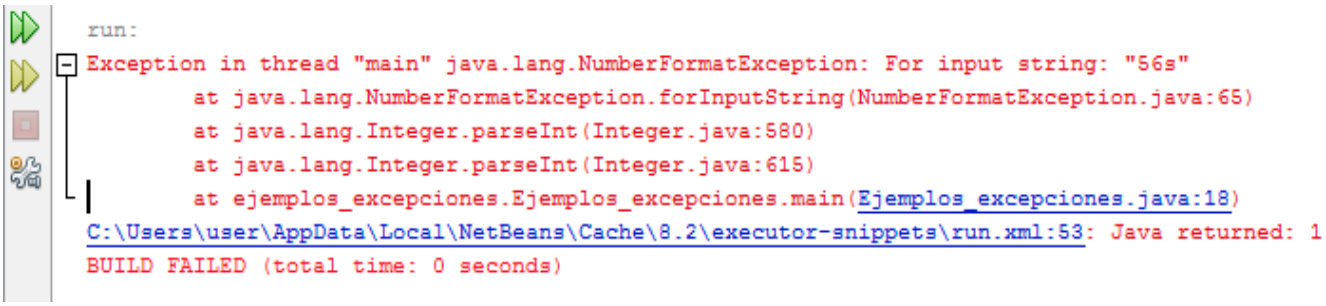


Lo que ha ocurrido es que la máquina virtual Java (el programa que ejecuta código Java) ha detectado una condición de error, la división por 0, y ha creado un objeto de la clase java.lang.ArithmeticException. Como el método donde se ha producido la excepción no es capaz de tratarla, la máquina virtual Java finaliza el programa en la línea 20 y muestra un mensaje de error con la información sobre la excepción que se ha producido.

Ejemplo: A continuación vamos a forzar una excepción de conversión, para ello vamos a intentar pasar a entero una cadena que no sólo lleva caracteres numéricos:

```
12  public class Ejemplos_excepciones {
13
14      public static void main(String[] args) {
15          String cadena = "56s";
16          int num;
17
18          num = Integer.parseInt(cadena);
19
20          System.out.println("El número es " + num);
21      }
22
23  }
```

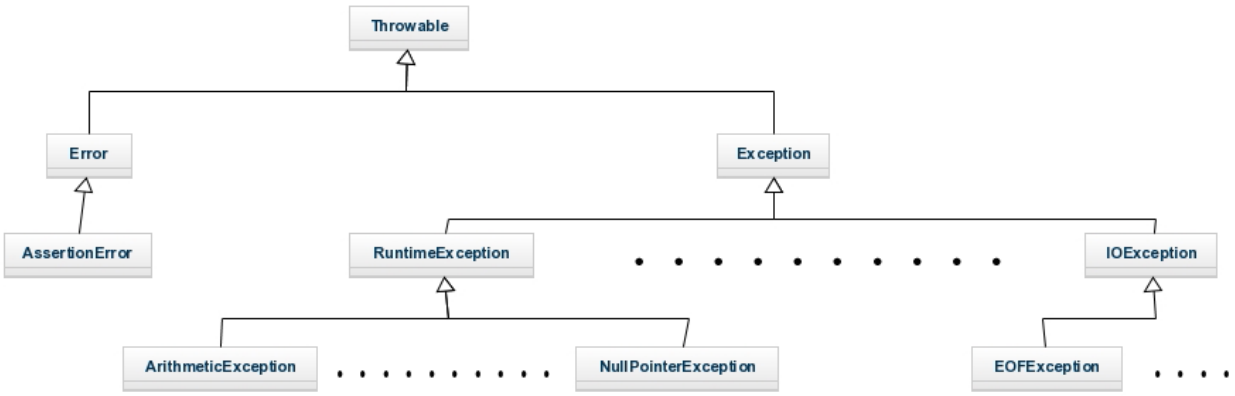
Siendo la salida:



Debido a que la cadena no tiene el formato adecuado ("56s" no representa un número válido), el método Integer.parseInt(...) no puede convertirla a un valor de tipo int y lanza la excepción NumberFormatException. La máquina virtual Java finaliza el programa en la línea 18 y muestra por pantalla la información sobre la excepción que se ha producido.

JERARQUÍA Y TIPOS DE EXCEPCIONES JAVA

La clase **Exception** hereda de Throwable, y a su vez, todas las excepciones heredan de Exception.



Como java.lang es importado de forma implícita en todos los programas, la mayor parte de las excepciones derivadas de RuntimeException están disponibles de forma automática.

Las Excepciones pueden ser comprobadas y no comprobadas:

- **Excepciones comprobadas**: aquellas que Java comprueba durante la compilación, antes de la ejecución del programa.
- **Excepciones no comprobadas**: aquellas que Java no puede comprobar durante la compilación y se producirán durante la ejecución del programa.

Las **excepciones comprobadas** definidas en java.lang son:

EXCEPCIÓN	SIGNIFICADO
ClassNotFoundException	No se ha encontrado la clase.
CloneNotSupportedException	Intento de duplicado de un objeto que no implementa la interfaz clonable.
IllegalAccessException	Se ha denegado el acceso a una clase.
InstantiationException	Intento de crear un objeto de una clase abstracta o interfaz.
InterruptedException	Hilo interrumpido por otro hilo.
NoSuchFieldException	El campo solicitado no existe.
NoSuchMethodException	El método solicitado no existe.

Las **subclases de RuntimeException no comprobadas** son:

EXCEPCIÓN	SIGNIFICADO
AritmeticException	Error aritmético como división entre cero.
ArrayIndexOutOfBoundsException	Índice de la matriz fuera de su límite.
ArrayStoreException	Asignación a una matriz de tipo incompatible.
ClassCastException	Conversión invalida.
IllegalArgumentException	Uso inválido de un argumento al llamar a un método.
IllegalMonitorStateException	Operación de monitor inválida, como esperar un hilo no bloqueado.
IllegalStateException	El entorno o aplicación están en un estado incorrecto.
IllegalThreadStateException	La operación solicitada es incompatible con ell estado actual del hilo.
IndexOutOfBoundException	Algún tipo de índice está fuera de su rango o de su límite.
NegativeArraySizeException	La matriz tiene un tamaño negativo.
NullPointerException	Uso incorrecto de una referencia NULL.
NumberFormatException	Conversión incorrecta de una cadena a un formato numérico.
SecurityException	Intento de violación de seguridad.
StringIndexOutBounds	Intento de sobrepasar el límite de una cadena.
TypeNotPresentException	Tipo no encontrado.
UnsupportedOperationException	Operación no admitida.

Es interesante conocer los distintos tipos de excepciones, pero no es necesario sabérselas de memoria ni entender exactamente cuándo se produce cada una.

Aunque las excepciones que incorpora Java, gestionan la mayoría de los errores más comunes, es probable que el programador prefiera crear sus propios tipos de excepciones para gestionar situaciones específicas de sus aplicaciones. Lo veremos en un apartado posterior.

MANEJAR EXCEPCIONES (TRY – CATCH – FINALLY)

MANEJADORES DE EXCEPCIONES

En Java se pueden manejar excepciones utilizando tres mecanismos llamados manejadores de excepciones. Existen tres y funcionan conjuntamente:

- Bloque **try** (intentar): código que podría lanzar una excepción.
- Bloque **catch** (capturar): código que manejará la excepción si es lanzada.
- Bloque **finally** (finalmente): código que se ejecuta tanto si hay excepción como si no.

Un **manejador de excepciones** es una bloque de código encargado de tratar las excepciones para intentar recuperarse del fallo y **evitar que la excepción sea lanzada descontroladamente hasta el main y termine el programa**.

Siempre que se utilice un try es obligatorio utilizar al menos un catch. El finally es opcional.

```
try {
        // Instrucciones que podrían lanzar una excepción.
}
catch (TipoExcepción nombreVariable) {
        // Instrucciones que se ejecutan cuando 'try' lanza una excepción.
}
finally {
        // Instrucciones que se ejecutan tanto si hay excepción como si no.
}
```

El **bloque try intentará ejecutar el código**. Si se produce una excepción se abandona dicho bloque (no se ejecutarán las demás instrucciones del try) y se saltará al bloque catch. Lógicamente, si en el try no se produce ninguna excepción el bloque catch se ignora.

El bloque catch capturará las excepciones del tipo TipoExcepción, evitando que sea lanzada al método que nos llamó. Aquí deberemos escribir las intrucciones que sean necesarias para manejar el error. Pueden especificarse varios bloques catch para distintos tipos de excepciones.

El bloque finally es opcional y se ejecutará tanto si se ha lanzado una excepción como si no.

Veamos un ejemplo sencillo: Vamos a producir una excepción y tratarla haciendo uso de los manejadores try-catch:

```
12  public class Ejemplos_excepciones {
13
14      public static void main(String[] args) {
15          int x = 1, y = 0;
16
17          try
18          {
19              int div = x / y;
20
21              System.out.println("La ejecución no llegará aquí.");
22          }
23          catch(ArithmeticException ex)
24          {
25              System.out.println("Has intentado dividir entre 0");
26          }
27
28          System.out.println("Fin del programa");
29      }
30  }
```

Siendo la salida:

```
run:
Has intentado dividir entre 0
Fin del programa
BUILD SUCCESSFUL (total time: 0 seconds)
```

Al intenta dividir por cero se lanza automáticamente una ArithmeticException. Como esto sucede dentro del bloque try, la ejecución del programa pasa al primer bloque catch porque coincide con el tipo de excepción producida (ArithmeticException). Se ejecutará el código del bloque catch y luego el programa continuará con normalidad.

PARTICULARIDADES DE LA CLÁUSULA CATCH

Es importante entender que el bloque catch solo capturará excepciones del tipo indicado. Si se produce una excepción distinta no la capturará. Sin embargo capturará excepciones heredadas del tipo indicado. Por ejemplo, catch (ArithmeticException e) capturará cualquier tipo de excepción que herede de ArithmeticException. El caso más general es catch (Exception e) que capturará todo tipo de excepciones porque en Java todas las excepciones heredan de Exception.

Sin embargo, es mejor utilizar excepciones lo más cercanas al tipo de error previsto, ya que lo que se pretende es recuperar el programa de alguna condición de error y si "se meten todas las excepciones en el mismo saco", seguramente habrá que averiguar después qué condición de error se produjo para poder dar una respuesta adecuada.

El objetivo de una cláusula catch es resolver la condición excepcional para que el programa pueda continuar como si el error nunca hubiera ocurrido.

CLÁUSULAS CATCH MÚLTIPLES

Se pueden especificar varias cláusulas catch, tantas como queramos, para que cada una capture un tipo diferente de excepción.

```
try {
        // instrucciones que pueden producir distintos tipos de Excepciones
}
catch (TipoExcepción1 e1) {
        // instrucciones para manejar un TipoExcepción1
}
catch (TipoExcepción2 e2) {
        // instrucciones para manejar un TipoExcepción2
}
catch (TipoExcepciónN eN) {
        // instrucciones para manejar un TipoExcepciónN
}
finally { // opcional
        // instrucciones que se ejecutarán tanto si hay excepción como si no
}
```

Cuando se lanza una excepción dentro del try, se comprueba cada sentencia catch en orden y se ejecuta la primera cuyo tipo coincida con la excepción lanzada. Los demás bloques catch serán ignorados. Luego se ejecutará el bloque finally (si se ha definido) y el programa continuará su ejecución después del bloque try-catch-finally.

Si el tipo excepción producida no coincide con ninguno de los catch, entonces la excepción será lanzada al método que nos llamó.

Veamos un ejemplo con cláusulas catch múltiples. Vamos a ejecutar distintas instrucciones dentro del bloque try y manejaremos las excepciones de división por cero y de si sobrepasamos el tamaño del vector.

```
14  public class Ejemplos_excepciones {
15
16      public static void main(String[] args) {
17          int x, y, div, pos;
18          int[] v = {1,2,3};
19          Scanner in = new Scanner(System.in);
20
21          try
22          {
23              System.out.print("Introduce el numerador: ");
24              x = in.nextInt();
25
26              System.out.print("Introduce el denominador: ");
27              y = in.nextInt();
28
29              div = x / y;
30
31              System.out.println("La división es " + div );
32
33              System.out.print("Introduce la posición del vector a consultar: ");
34              pos = in.nextInt();
35
36              System.out.println("El elemento es " + v[pos] );
37
38          }
39          catch(ArithmeticException ex)
40          {
41              System.out.println("División por cero: " + ex);
42          }
43          catch(ArrayIndexOutOfBoundsException ex)
44          {
45              System.out.println("Sobrepasado el tamaño del vector: " + ex);
46          }
47
48          System.out.println("Fin del programa");
49      }
50  }
```

Pueden suceder tres cosas diferentes:

- El try se ejecuta sin excepciones, se ignoran los catch y se imprime “Fin del programa”.
- Se produce la excepción de división por cero (línea 29), el flujo de ejecución salta al 1er catch, se imprime el mensaje “División por cero...” y luego “Fin del programa”.
- Se produce la excepción de sobrepasar el vector (línea 36), el flujo de ejecución salta al 2º catch, se imprime el mensaje “Sobrepasado el tamaño del vector...” y “Fin del programa”.

EL OBJETO EXCEPTION

Toda excepción genera un objeto de la clase Exception (o uno más específico que hereda de Exception). Dicho objeto contendrá detalles sobre el error producido. Puede ser interesante mostrar esta información para que la vea el usuario (que sepa qué ha sucedido) o el desarrollador (para depurar y corregir el código si es pertinente). En la cláusula catch tenemos acceso al objeto en caso de que queramos utilizarlo:

Los dos métodos de Exception más útiles son:

- **getMessage()** → Devuelve un String con un texto simple sobre el error.
- **printStackTrace()** → Es el que más información proporciona. Indica qué tipo de Excepción se ha producido, el mensaje simple, y también toda la pila de llamadas. Esto es lo que hace Java por defecto cuando una excepción no se maneja y acaba parando el programa.

```
...
catch (Exception e){
        // Mostramos el mensaje de la excepción
        System.err.println("Error: " + e.getMessage());
        // Mostramos toda la información, mensaje y pila de llamadas
        e.printStackTrace();
}
...
```

Los objetos de tipo Exception tienen sobrecargado el método toString() por lo que también es posible imprimirlos directamente mediante println().

```
...
catch (Exception e){
        // Mostramos el mensaje de la excepción
        System.out.println(e);
}
...
```

LANZAR EXCEPCIONES (THROW)

¿POR QUÉ LANZAR EXCEPCIONES?

Un programador puede programar su código de forma que se lancen excepciones cuando se intente hacer algo incorrecto o inesperado (en ocasiones es recomendable). Por ejemplo, cuando los argumentos que se le pasan a un método no son correctos o no cumplen ciertos criterios.

Téngase en cuenta que las excepciones pueden manejarse y controlarse sin que el programa se pare. Es decir, lanzar una excepción no implica necesariamente que el programa terminará, es simplemente es una forma de avisar de un error. Quien llame al método es responsable de manejar la excepción para que el programa no se pare.

CÓMO LANZAR UNA EXCEPCIÓN

Para lanzar la excepción se utiliza la palabra reservada throw seguido de un objeto de tipo Exception (o alguna de sus subclases como ArithmeticException, NumberFormatException, ArrayIndexOutOfBoundsException, etc.). Como las excepciones son objetos, deben instanciarse con "new". Por lo tanto, podemos lanzar una excepción genérica así:

```
throw new Exception();
```

Esto es equivalente a primero instanciar el objeto Exception y luego lanzarlo:

```
Exception e = new Exception();
throw e;
```

El constructor de Exception permite (opcionalmente) un argumento String para dar detalles sobre el problema. Si la excepción no se maneja y el programa se para, el mensaje de error se mostrará por la consola (esto es muy útil para depurar programas).

```
throw new Exception("La edad no puede ser negativa");
```

En lugar de lanzar excepciones genéricas (Exception) también es posible lanzar excepciones específicas de Java como por ejemplo ArrayIndexOutOfBoundsException, ArithmeticException, NumberFormatException, etc. En Java todas las clases de excepciones heredan de Exception.

```
throw new NumberFormatException("...");
```

En lugar de lanzar excepciones propias de Java (ArithmeticException, NumberFormatException, ArrayIndexOutOfBoundsException, etc.) normalmente **es preferible lanzar excepciones genéricas 'Exception'** o mejor aún, utilizar nuestras propias excepciones.

INDICAR LA EXCEPCIÓN EN LA CABECERA DEL MÉTODO

Es obligatorio indicar en la cabecera del método que puede lanzar excepciones. Para ello hay que añadir, a la derecha de la cabecera y antes de las llaves, la palabra reservada throws seguido del tipo de excepción que puede lanzar (si puede lanzar distintos tipos de excepciones deben indicarse todas separadas por comas).

Por ejemplo, veamos el método setEdad(int edad) de la clase Persona. Como hemos decidido que la edad de una persona no puede ser negativa, lanzaremos una excepción si edad < 0.

```
// Setter del atributo edad. Debe ser >= 0
public void setEdad(int edad) throws Exception {
        if (edad < 0)
                throw new Exception("Edad negativa no permitida");
        else
                this.edad = edad;
        }
}
```

Hay que tener en cuenta que al lanzar una excepción se parará la ejecución de dicho método (no se ejecutará el resto del código del método) y se lanzará la excepción al método que lo llamó. Si por ejemplo desde la función main llamamos a setEdad(), como puede suceder que setEdad() lance una excepción, entonces en la práctica es posible que el main lance una excepción (no directamente con un throw, sino por la excepción que nos lanza setEdad()). Por lo tanto, también tenemos que especificar en el main que se puede lanzar una excepción:

```
public static void main(String[] args) throws Exception {

        setEdad(27); // Puede lanzar una excepción

}
```

LANZANDO DISTINTOS TIPOS DE EXCEPCIONES

Un método puede lanzar distintos tipos de excepciones (si lo consideramos necesario). En tal caso hay que especificar todos los tipos posibles en la cabecera, separados por comas. Por ejemplo, imaginemos que el constructor de Persona toma como argumentos el dni y la edad, y queremos lanzar excepciones distintas según cada caso.

```
public Persona(String dni, int edad) throws InvalidDniException, InvalidEdadException
{
        if (!dni.matches("^[0-9]{8}[A-Z]")) {
                throw new InvalidDniException("DNI no válido: " + dni);
        }
        if (edad < 0) {
                throw new InvalidEdadException("Edad no válida: " + edad);
        }
        this.dni = dni;
        this.edad = edad;
}
```

Ojo, téngase en cuenta que InvalidEdadException e InvalidDniException no son tipos de excepciones de Java, sino excepciones propias que nosotros habríamos definido a medida para nuestro software. Posteriormente veremos como definirlas.

LAS EXCEPCIONES SE LANZAN DE MÉTODO A MÉTODO

Observese que el lanzamiento de excepciones se produce recursivamente a través de la secuencia de llamadas a métodos. Imaginemos que en el método A llamamos al método B, que llama al método C, etc. hasta llegar a E.

A → B → C → D → E (secuencia de llamadas de métodos)

Si el método E lanza una excepción, esta le llegará a D que a su vez se la lanzará a C, etc. recorriendo el camino inverso hasta llegar al método inicial A.

A ← B ← C ← D ← E (secuencia de lanzamiento de excepciones)

Por lo tanto, como todos estos métodos pueden acabar lanzando una excepción, en sus cabeceras habrá que incluir el throws Exception (o el que corresponda según el tipo de excepción).

¡Ojo! No es recomendable dejar que las Excepciones de nuestro software lleguen de forma descontrolada hasta el main y terminen el programa. Lo ideal es manejar las excepciones.

DEFINIR EXCEPCIONES PROPIAS

Cuando desarrollamos software, sobre todo al desarrollar nuestras propias clases, es habitual que se puedan producir excepciones que no estén definidas dentro del lenguaje Java. Para crear una excepción propia tenemos que definir una clase derivada de la clase base Exception. El uso de esta nueva excepción es el mismo que hemos visto.

Ejemplo: Vamos a ver un ejemplo sencillo de definición y uso de un nuevo tipo de excepción llamada ExcepcionPropia que utilizaremos cuando a se le pase a 'método' un valor superior a 10.

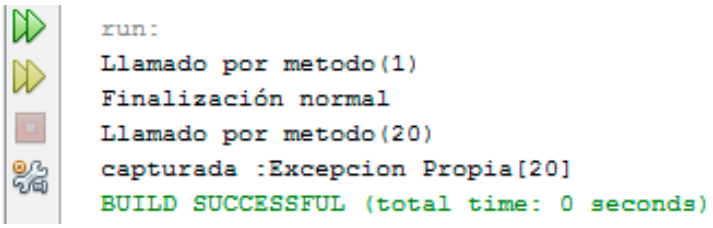
El main y el método que lanza la excepción:

```
14    public class Ejemplos_excepciones {
15
16        public static void main(String[] args) {
17            try
18            {
19                metodo(1);
20                metodo(20);
21            }
22            catch (ExcepcionPropia e)
23            {
24                System.out.println("capturada :" + e);
25            }
26        }
27
28        static void metodo(int n) throws ExcepcionPropia
29        {
30            System.out.println("Llamado por metodo(" + n +")");
31
32            if (n > 10)
33                throw new ExcepcionPropia(n);
34
35            System.out.println("Finalización normal");
36        }
37    }
```

La definición de la nueva excepción.

```
12    public class ExcepcionPropia  extends Exception
13    {
          private int num;
15
16        ExcepcionPropia(int n)
17        {
18            this.num = n;
19        }
          public String toString()
21        {
22            return "Excepcion Propia[" + this.num + "]";
23        }
24
25    }
```

Siendo la salida:



Como se puede observar la excepción se lanza cuando el número es mayor que 10 y será tratada por la nueva clase creada que hereda de Exception. Además se sobreescribe el método toString que es el encargado de mostrar el mensaje asociado a la excepción.